

설계서 — 웹앱 MVP 우선

작업명: 개인정보를 받지 않는 학교 기반 소셜 투표 웹서비스 MVP 구축, 그리고 그 위의 데이터 파이프라인

이 문서는 Claude Code에서 구현할 때 참조하는 **계획서**다. 구현 상세(코드, 스키마 정의)는 여기 적지 않는다. 프로젝트 요약은 [CLAUDE](#), 결정 이력은 [DECISIONS](#) 참조.

개정 이력

- 2026-07-29 v1 — 파이프라인 단독 설계
- 2026-07-29 v2 — **웹앱을 최우선 트랙으로 재편**. v1은 접속해서 볼 수 있는 화면이 진행 순서에 없다는 결함이 있었다. 아울러 인증을 개인정보 없는 익명 방식으로 재설계.
- 2026-07-30 v3 — W8(학교 정보 화면) 추가, 성별 수집 결정, 학생용 내용/성인 이용자로 정리. 진행 순서 표를 실제 상태에 맞춤.

1. 작업 컨텍스트

1.1 배경

기존 학교 소셜 투표 서비스의 DB(가입자 67.7만, 14개월치)를 분석하며 **분석을 가로막는 구조적 결함**이 반복 확인됐다. 탈퇴 테이블에 유저 식별자가 없고, 하트 원장이 잔액을 설명하지 못하고, 관계 39개 중 FK 선언은 17개뿐이었다.

그 결함을 담은 스키마를 새로 설계했고(MVP 기준 42 테이블, FK 77), 합성 데이터 786만 행으로 정합성을 검증했다. **여기까지가 완료된 상태다**.

이제 그 스키마 위에 **실제로 접속해서 쓸 수 있는 웹앱**을 올린다.

1.2 목적

1. 지인이 링크로 접속해 **실제로 투표할 수 있는 웹앱**을 만든다
2. **개인정보를 일절 받지 않는다** — 이메일·전화번호·실명·비밀번호 없음
3. 그렇게 쌓인 실데이터와 합성 데이터를 파이프라인으로 BigQuery에 적재한다

1.3 범위

MVP에 포함

영역	내용
인증	익명 계정 자동 생성. 아이디·비번·이메일 없음
온보딩	닉네임 정하기, 성별 선택, 소속(학교·반) 선택
친구	초대 코드 를 주고받아 추가. 5명 이상이면 투표 해금
투표	질문 출제 → 후보 4명 → 선택. CLASS/SCHOOL/GLOBAL 세 스코프
셔플	1회 한정. MVP에서는 광고를 스텝 으로 대체(3초 대기)
받은 투표	알림 확인, 하트로 힌트 구매, 답변 공개
하트	가입 지급·투표 적립·힌트 차감. 충전(결제)은 제외

MVP에서 제외

- **실결제·실광고** — 계정 개설과 심사가 필요해 MVP 범위 밖. 스키마는 이미 준비돼 있다
- **익명 게시판** — 신고 검토 인력이 없으면 열지 않는다([DECISIONS](#)에 재검토 조건)
- **급식·시간표·공지·학사일정** — MVP(W0~W7) 화면에서 제외.
데이터는 P3에서 모으고 **화면은 W8**에서 붙인다. RLS는 이미 열려 있다
- **연락처 동기화** — 웹은 연락처 접근이 불가하고, 전화번호는 개인정보라 받지 않는다
- **신고·제재 화면** — 데이터 구조만 두고 운영은 수동.
게시판 신고 버튼은 W9 에서 붙였으나, 검토·제재는 여전히 사람이 DB 를 본다

1.4 개인정보를 받지 않는다는 것의 의미

이 프로젝트의 핵심 제약이다.

받지 않는 것	대신 쓰는 것
이메일	익명 계정(브라우저에 귀속)
비밀번호	없음. 로그인 절차 자체가 없다
전화번호	없음. 친구 추가는 초대 코드로

받지 않는 것	대신 쓰는 것
실명	유저가 직접 정하는 닉네임

유저가 입력하는 정보는 닉네임·성별·소속(학교·반)이다.

성별은 힌트(받은 투표 1단계)로 파는 정보라 비워둘 수 없다.

⚠ 다만 "닉네임 + 성별 + 학교 + 반" 조합은 소규모 집단에서 **개인을 특정할 수 있다**.
법적으로 개인정보가 아니라고 단정할 수 없으므로, 개인정보처리방침에 수집 항목과 목적을 명시한다. 성인 지인 대상 클로즈드 테스트라는 전제도 유지한다.

1.5 제약조건

구분	제약
작업자	개발 경험 없음. 코드 작성·디버깅은 전량 에이전트가 수행
비용	월 0~3만원. 무료 티어를 벗어나는 선택지는 채택하지 않음
개인정보	위 1.4 준수. 성인 지인 20~50명 클로즈드 테스트
보안	RLS 없이는 실유저를 받지 않는다. W1의 완료 조건
데이터 분리	실유저(Supabase)와 합성(로컬 Postgres)을 섞지 않음
기존 자산	mysql 컨테이너의 final / hackle 은 읽기만

1.6 용어 정의

용어	의미
익명 계정	이메일·비번 없이 브라우저에 귀속되는 계정. Supabase Anonymous Sign-in
초대 코드	유저마다 발급되는 짧은 코드. 이걸 주고받아 친구를 맺는다
스코프	투표 후보 범위. CLASS / SCHOOL / GLOBAL . 셋 다 친구 안에서 의 범위다
아이템	출제된 질문 1건
세션	아이템 여러 개를 묶은 한 번의 투표 회차
셔플	후보 4명이 마음에 안 들 때 1회 한정 재추첨

항목	내용
제외	user_contact 테이블은 MVP DDL에서 생성하지 않음(전화번호 해시 보관 = 개인정보)
성공 기준	변경 DDL 적용 후 합성 데이터 재생성·재적재·정합성 17종 통과
실패 처리	마이그레이션 오류 → 즉시 수정. 기존 합성 데이터는 언제든지 재생성 가능하므로 손실 없음

W1 · Supabase 구축 + RLS — 보안 게이트

항목	내용
입력	조정된 DDL
출력	Supabase 프로젝트, RLS 정책
성공 기준	anon 키로 남의 데이터를 못 읽는 것을 실제로 시험해 확인
검증 방법	유저 A의 토큰으로 유저 B의 투표·하트·친구 목록 조회 시도 → 전부 빈 결과
실패 처리	하나라도 뚫리면 다음 단계로 넘어가지 않는다

RLS는 선언만으로 끝나지 않는다. P0에서 제약을 실제로 위반시켜 검증했듯, RLS도 **실제로 남의 데이터를 훔쳐보려 시도해서** 막히는지 확인한다.

W2 · 앱 뼈대

Next.js 프로젝트, 라우팅, Supabase 클라이언트 연결, 익명 로그인.

성공 기준	링크 접속 → 자동으로 계정 생성 → 새로고침해도 유지
-------	--------------------------------

W3 · 온보딩

닉네임 입력, 성별 선택, 학교·반 선택, 초대 코드 발급.

성공 기준	온보딩 완료 후 app_user 행이 생성되고 invite_code 가 화면에 보인다
-------	---

W4 · 친구

초대 코드 입력으로 친구 추가, 친구 목록, 5명 게이트 해금.

성공 기준	서로 다른 브라우저 두 개로 코드를 교환해 친구가 맺어진다 · 5명째에 service_unlocked_at 이 찍힌다
-------	---

W5 · 투표 — 가장 복잡한 단계

질문 출제, 후보 4명 추출, 셔플, 선택 제출, 하트 적립.

성공 기준	세 스코프별로 후보가 규칙대로 뽑힌다 · 셔플이 1회만 된다 · 하트가 정확히 적립된다
검증 방법	투표 후 qa/checks/integrity.sql 을 실유저 데이터에 돌려 위반 0건

후보 풀 4명 미만 — 스코프는 그대로 두고 친구 중 다른 사람으로 채운다.
채운 인원은 vote_item.padded_count 에 남고 화면에도 밝힌다. 친구 전체로도 4명이 안 되면 그 질문은 내지 않는다. (2026-07-30 결정, [DECISIONS](#))

W6 · 받은 투표

알림 목록, 힌트 구매(하트 차감), 답변 공개. 내가 한 투표 목록도 같은 화면의 탭으로 둔다 — 가려야 하는 것은 "누가 나를 뽑았나"이지 "내가 누구를 뽑았나"가 아니다.

힌트는 4단계 누진이다: **성별 200** → **초성 300** → **반 500** → **전체 공개 1000**
(구 서비스 실측 요금). 좁히는 폭이 작은 것부터 연다.

성공 기준	하트가 모자라면 구매가 막힌다 · 원장과 잔액이 일치한다
-------	---------------------------------

W7 · 배포 + 클로즈드 테스트

Vercel 배포, 개인정보처리방침, 지인 초대.

성공 기준	외부인이 링크로 접속해 투표까지 완료 · 데이터가 Supabase에 정상 기록
-------	---

W8 · 학교 정보 — P3 이후

급식·시간표·공지·학사일정을 앱에서 본다. 투표 서비스가 아니라 "학교 앱"으로서의 몫이고, 매일 열어볼 이유를 만드는 화면이다.

성공 기준	내 학교·반의 오늘 급식과 시간표가 보이고, 공지를 읽으면 읽음 처리된다
-------	--

왜 W7 뒤편가 — 두 가지가 먼저 있어야 한다.

1. **데이터.** NEIS에서 받아와야 한다(P3). 그전에는 테이블이 비어 있다.
2. **실제 학교.** 지금 조직은 클로즈드 테스트용 임시 데이터("코드잇 DA 14기")라 급식도 시간표도 존재하지 않는다. 실제 학교 목록으로 교체된 뒤라야 화면이 의미를 갖는다.

DB는 이미 준비돼 있다. meal_plan meal_menu_item timetable school_notice school_event school_notice_read 테이블과 RLS 정책(자기 학교·자기 반 것만)이 W1에서 함께 만들어졌다. W8에서 새로 할 일은 화면과 조회 코드다.

이 단계가 로드맵에서 빠져 있었다(2026-07-29 발견). P3가 "데이터를 모은다"까지만 다루고 "그 데이터를 어디서 보여주나"가 없었다. 스키마와 RLS는 화면을 전제로 설계돼 있었으므로, 빠진 것은 계획이지 설계가 아니다.

아직 단계가 배정되지 않은 기능

작업자가 하겠다고 밝혔지만 **순서가 정해지지 않은 것들**이다.
여기 적어두는 이유는 "결정은 있는데 단계가 없어" 잊히는 일을 막기 위해서다 (W8 학교 정보가 실제로 그렇게 빠져 있었다).

기능	스키마	상태
자유게시판	있음	W9 에서 열었다 (아래)
지목한 사람에게 익명 메시지	없음	받은 투표에서 "나를 뽑은 사람"에게 보낸다. 보내는 쪽은 상대가 누군지 모른다
채팅방 열기 (하트 차감)	없음	하트 소비처가 힌트 하나뿐인데 둘이 되면 하트 경제 분석이 풍부해진다

남은 둘은 같은 전제를 공유한다 — 자유 텍스트가 사람에게 직접 간다.

익명 게시판을 v1에서 뺀 이유(신고를 검토할 사람이 없다)가 그대로 적용되고,
1:1 메시지는 공개 게시판보다 위험하다. 공개된 글은 제3자가 신고할 수 있지만 사적인 메시지는 받은 사람이 말하기 전까지 아무도 모른다.

그래서 이 둘을 열기 전에 **차단 화면이 먼저 있어야 한다.**

신고는 W9 에서 붙였지만 block_record 는 여전히 테이블만 있다.
게시판에서는 "안 보이게"보다 "내려가게"가 먼저였으나, 1:1 메시지는 반대다.

W9 · 자유게시판

익명이 아니라 **닉네임이 드러나는** 형태다. 범위는 같은 학교.

항목	내용
기능	목록 · 작성 · 상세 · 댓글 · 좋아요 · 신고
제외	대댓글(화면 복잡도), 카테고리 분리(20~50명이 5개로 나뉘면 전부 빈다)
성공 기준	같은 학교 안에서만 보이고, 쓰기가 RPC 밖으로 새지 않는다
검증	db/rls/verify.py 의 게시판 29종 (전체 115항목)

왜 익명이 아닌가 — 익명 게시판을 뺀 이유가 "사고가 나도 책임을 물을 수 없다"였다. 글쓰기가 붙으면 그 전제가 바뀐다. 댓글도 마찬가지로 익명 번호를 쓰지 않는다.

신고는 자동으로 아무 일도 하지 않는다. 자동 숨김은 집단 신고에 취약해 채택하지 않았다([DECISIONS](#)). 기록만 남고 작업자가 보고 판단한다.

2.3 LLM 판단 영역 vs 코드 처리 영역

이 서비스에는 LLM 이 들어가지 않는다. 이 절은 *프로젝트를 만드는 과정에서* 무엇을 에이전트가 판단하고 무엇을 스크립트로 굳힐지에 대한 것이다.

이용자가 앱을 쓸 때 호출되는 AI 는 없다.

결정론적이고 반복되는 것은 코드, 맥락과 판단이 필요한 것은 LLM.

코드가 처리

DDL 실행 · 합성 데이터 생성 · 적재 · 정합성 검사 쿼리 · 후보 추출 · 하트 계산 · 스케줄링

LLM이 판단

판단 지점	무엇을 판단하나	왜 코드로 안 되나
스키마 변경 필요성	화면을 만들다 발견된 요구가 컬럼 추가인지 테이블 분리인지	설계 트레이드오프
후보 풀 부족 처리	스코프를 낮출지, 질문을 건너뛸지	UX와 데이터 정합성의 균형
RLS 정책 설계	어느 테이블에 어떤 조건을 걸지	기능 요구와 보안의 절충
합성 데이터 현실성	분포가 그럴듯한지	"그럴듯함"에 고정 기준이 없음

판단 지점	무엇을 판단하나	왜 코드로 안 되나
품질 이상의 성격	버그인지 정상 변동인지	맥락 의존
에스컬레이션 여부	작업자에게 물어야 할 사안인지	사안의 중대성 평가

원칙: LLM이 매번 같은 판단을 반복하고 있다면, 규칙으로 굳혀 코드로 내려야 한다는 신호다.

2.4 실패 처리 분기

상황	처리
RLS 정책이 하나라도 뚫림	즉시 중단. 실유저를 받지 않는다
하트 원장과 잔액 불일치	즉시 중단 + 에스컬레이션. 구 시스템 최대 결함의 재발
FK 위반	즉시 중단 + 에스컬레이션
후보 풀 4명 미만	다른 친구로 채움 (padded_count 기록) → 친구 전체로도 부족하면 스킵
Supabase 무료 티어 한도 초과	에스컬레이션 — 비용 결정은 작업자 몫
NEIS API 타임아웃/5xx	재시도 3회, 지수 백오프
NEIS API 4xx	에스컬레이션 (키 만료 등)
급식 데이터 없음(주말·방학)	스킵 , 로그만
합성 데이터 제약 위반	로직 수정 후 전량 재생성

에스컬레이션 형식 — 무슨 일이 있었는가 / 왜 자동 처리하지 않았는가 / 선택지와 결과 / 권장안.

3. 구현 스펙

구조 개요만 정의한다. 실제 코드는 각 단계 착수 시 작성한다.

3.1 폴더 구조

```
ping-v2/
├─ CLAUDE.md          # 세션마다 로드되는 컨텍스트
├─ DECISIONS.md       # 결정 이력
├─ README.md
├─ .gitignore / .env.example / requirements.txt
├─ docs/
│   └─ design-spec.md # 이 문서
├─ web/               # Next.js 웹앱
│   └─ src/
│       ├── app/      # 라우트 (/ , /privacy)
│       ├── components/ # 온보딩·친구·투표·받은투표 화면
│       └─ lib/        # Supabase 클라이언트, 도메인별 호출
├─ db/
│   ├── ddl/          # 테이블 생성 SQL
│   ├── rls/          # ★ 신규 - RLS 정책
│   └─ migrations/
├─ generator/         # 합성 데이터 생성
├─ pipeline/          # ★ P4 - Postgres → BigQuery 적재
│   ├── tables.yaml   # 테이블별 적재 방식 (full 12 / incremental 30)
│   ├── extract_load.py
│   └─ verify_load.py # 행 수 대조
├─ airflow/           # docker-compose + ping_raw_load DAG
├─ bigquery/          # stg / mart 변환 SQL (P6)
├─ qa/checks/
└─ data/synthetic/
```

3.2 인증 구조 (개인정보 없음)

브라우저 첫 접속

```
|
├─ Supabase Anonymous Sign-in → auth.users 에 uuid 생성 (이메일·비번 없음)
|
├─ 온보딩: 닉네임 + 학교·반 선택
|
└─ app_user 행 생성 (auth_user_id = 그 uuid, invite_code 자동 발급)
```

알려진 한계 — 익명 계정은 브라우저에 귀속된다. 캐시·쿠키를 지우면 계정이 사라진다.
MVP에서는 이를 감수하고, 안내 문구로 알린다. 복구 코드 기능은 v2로 미룬다.

3.3 스키마 변경 (W0)

대상	변경	이유
app_user.phone_hash	제거	전화번호는 받지 않는다
app_user.name_masked	→ nickname	실명 마스킹이 아니라 유저가 정한 별명
app_user.invite_code	추가 (UNIQUE)	친구 추가의 유일한 수단
user_contact	MVP 미생성	전화번호 해시 보관 = 개인정보
friend_recommendation.reason	MUTUAL_CONTACT 미사용	연락처를 안 받으므로
friend_request.source	INVITE_CODE 추가	유입 경로 추적

3.4 작업 방식 — 서버에이전트는 만들지 않았다

이 서비스에는 **LLM 이 들어가지 않는다**. 이용자가 앱을 쓸 때 호출되는 AI 는 없다.
아래는 *이 프로젝트를 만드는 방식*에 대한 이야기다.

초기 설계에는 서버에이전트 3종(data-generator · pipeline-builder · data-qa)을 두고 합성 데이터·DAG·품질검사를 위임할 계획이 있었다. **P0~P4 를 끝낸 지금까지 하나도 만들지 않았고, 필요해진 적도 없다.**

위임이 이득이 되려면 **넘길 일이 충분히 크고, 판단이 거의 안 섞여 있어야** 한다.
그런데 실제로 해보니 이 프로젝트의 일은 그렇지 않았다.

위임하려던 것	실제로 어땠나
합성 데이터 생성기	하트 원장의 잔액 시뮬레이션이 핵심이었다. balance_after >= 0 을 지키며 유저별 타임라인을 걷는 부분은 스키마 제약을 이해해야 쓸 수 있는 코드다
Airflow DAG	결과물이 30줄이다. 어려운 것은 DAG 이 아니라 증분 키를 무엇으로 삼을지 였고, 그건 위임 대상이 아니다
품질 검사 실행	명령 한 줄이다. 어려운 것은 결과가 진짜 문제인지 판단 하는 쪽인데, 원래 위임하지 않기로 한 부분이다

세 경우 모두 **"쉬운 부분은 이미 스크립트가 하고, 남은 것은 판단"** 이었다.
컨텍스트를 넘기는 비용이 얻는 것보다 컸다.

그래서 유지하는 방식은 이렇다 — **메인이 직접 하고, 반복되는 것은 에이전트가 아니라 스크립트로 굳힌다.** 실제로 그렇게 쌓인 것이 아래 스크립트 표다.

verify.py (RLS 86항목)와 verify_load.py (행 수 대조)가 대표적이다.

사람이든 에이전트든 매번 판단하지 않아도 되도록 **규칙을 코드로 내린 것이다.**

다시 볼 조건 — 위임할 일이 **혼자 30분 이상 걸리고, 중간에 설게 판단이 없을 때.**

지금까지 그런 일은 없었다.

3.5 스크립트

/gen-data /check-rls 같은 슬래시 명령을 만들 계획도 있었으나 만들지 않았다.

스크립트 이름을 그대로 부르는 것과 차이가 없었고, 이름이 하나 더 생기면

"진짜 무엇이 실행되는가"가 한 겹 가려진다. 개발 경험이 없는 작업자에게는 그 한 겹이 그대로 비용이다.

스크립트	역할
db/apply.py	DDL + 마이그레이션 적용. 로컬 DB 도 이걸로만 만든다
generator/generate.py	합성 데이터 생성
generator/load.py	CSV → Postgres 적재 + 시퀀스 재동기화 + 워터마크 되돌리기
pipeline/extract_load.py	Postgres → BigQuery raw 적재
pipeline/verify_load.py	원천 ↔ BigQuery 행 수 대조
qa/checks/integrity.sql	정합성 17종
db/rls/verify.py	RLS 침투·동작 시험 128항목

3.6 산출물 형식

산출물	형식	비고
DDL	.sql	도메인별 1파일, 접두 번호로 실행 순서
RLS 정책	.sql	테이블별 정책 묶음
합성 데이터	.csv (UTF-8)	git 제외
생성 파라미터	.yaml	코드와 분리

산출물	형식	비고
품질 리포트	.md	날짜별
웹앱	Next.js (TypeScript)	Vercel 배포
DAG	.py	도메인별 1파일

4. 진행 순서

순서	단계	완료 조건	상태
—	P0 스키마·DDL	제약 위반 차단 검증 (W0 조정 후 42 테이블)	완료
—	P1 합성 데이터	786만 행, 정합성 17종 통과	완료
—	P2 Postgres 적재	행 수 일치 + 시퀀스 재동기화	완료
1	W0 스키마 조정	익명 인증 대응, 재생성·재적재 통과	완료
2	W1 Supabase + RLS	침투 시험 통과 — 남의 데이터가 안 보임	완료
3	W2 앱 뼈대	접속 시 익명 계정 자동 생성	완료
4	W3 온보딩	닉네임·성별·소속 저장, 초대 코드 발급	완료
5	W4 친구	코드 교환으로 친구 맺기, 5명 게이트	완료
6	W5 투표	세 스코프 후보 추출, 셔플 1회, 하트 적립	완료
7	W6 받은 투표	힌트 구매, 잔액 부족 시 차단	완료
8	W7 배포	외부인이 링크로 투표 완료	완료 (초대는 남음)

순서	단계	완료 조건	상태
9	P3 NEIS 수집	학교·학급·급식 완료 · DAG 화는 남음	일부
10	P4 BigQuery 적재	42테이블 789만 행 · 원천별 행 수 대조 통과	완료
11~13	P5 품질검증 → P6 stg/mart → P7 대시보드		

P6 의 첫 작업은 대리키다. 두 원천의 id 가 실제로 겹친다

(app_user 16개, vote_item 26개). raw 를 JOIN ... USING(id) 로 조인하면
실유저의 투표에 합성 유저의 프로필이 붙고, **오류는 나지 않는다.**

stg 층에서 _source || '-' || id 형태의 키를 만들어 이 실수를 구조적으로 막는다.

| 14 | W8 학교 정보 | 급식표 **완료** · 시간표·공지는 남음 |

| 15 | **W9 자유게시판** | 닉네임 노출 · 학교 단위 · 신고 포함 | **완료** |

| 16 | **W10 친구 추천** | 같은 학교 목록 → 요청 / 안 볼래 | **완료** |

예상 기간 — W0~W7까지 주 10~15시간 기준 **3~5주**. 가장 오래 걸리는 단계는 W5(투표)다.

P4 에서 실제로 한 것 — 증분 키가 테이블마다 다른 문제를 먼저 단았다.

created_at / served_at / started_at / synced_at 이 섞여 있었고
vote_candidate 는 시간 컬럼이 아예 없었다. 30개 테이블에 updated_at 과
트리거를 심어 적재 조건을 한 줄로 통일했다([DECISIONS](#)).

실유저와 합성 데이터는 is_synthetic 이 아니라 **_source 컬럼**으로 구분한다 —
그 플래그는 app_user 에만 있어서 나머지 41개 테이블에 쓸 수 없고, 두 원천의
id 가 둘 다 1부터라 충돌하는 문제도 풀지 못한다.

다음 작업: 지인 초대(W7 잔여) 또는 순서 11 — P5 품질 검증.

초대를 미룰 이유는 없어졌다. 증분이 처음부터 흐르므로 소급 적재가 필요 없다.